

6 EE PROJECTS THAT ONLY EXIST BECAUSE AI EXISTS

Circuits that fight back at the hardware level.

Analog design. RF. Signal processing. Crypto hardware. Real EE.

1. AI Voice Cloning Detector — analog filter bank catches what ears can't
2. EMI Side-Channel Sniffer — read what a processor is running from its radiation
3. Hardware AI Kill Switch — physically disconnect AI when behavior deviates
4. Deepfake-Proof Identity Badge — cryptographic proof of physical presence
5. Neuromorphic Sensor Fuser — analog circuits that compute like neurons
6. RF Fingerprinting Authenticator — identify devices by manufacturing imperfections

By @1nska • Electrical Engineer

PROJECT 1

AI Voice Cloning Detector

Analog circuit that detects AI-generated voices — no software, no ML model

Cost: ~\$15

Difficulty:
Intermediate-Advanced

Build Time: 1-2 weekends

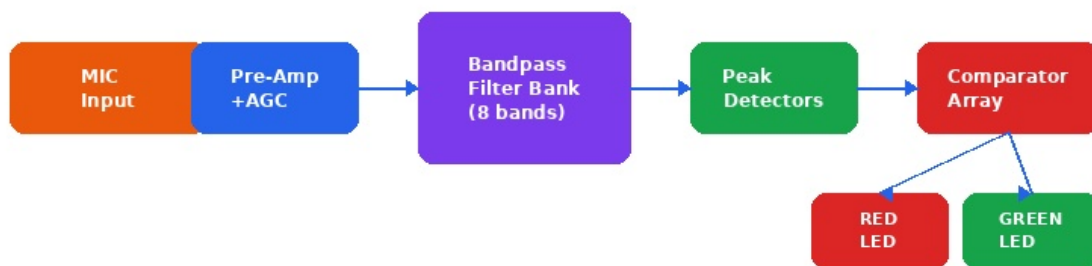
Why This Matters

AI voice cloning has reached the point where human ears can't tell the difference. But cloned audio has measurable artifacts: spectral energy gaps in the 2-4kHz range, unnatural phase continuity, and missing micro-variations that exist in real vocal cords. This project detects those artifacts using pure analog signal processing — no machine learning model, no microcontroller.

The key insight: Real voices have chaotic, broadband energy with natural spectral variation. AI clones have suspiciously smooth spectral envelopes because the synthesis model interpolates between training samples. An analog filter bank can measure this difference in real time.

System Architecture

AI Voice Cloning Detector — Signal Chain



Real voice → consistent spectral energy across bands → GREEN
AI clone → spectral gaps in 2-4kHz range + phase artifacts → RED

How It Works

- **Microphone + pre-amp:** Electret mic with JFET buffer, then an AGC stage (LM386 or OPA344) to normalize input levels regardless of speaker volume.
- **Bandpass filter bank (8 bands):** Eight parallel 2nd-order Sallen-Key bandpass filters covering 200Hz to 8kHz. Each band is ~1 octave wide. Center frequencies: 250, 500, 1K, 1.5K, 2K, 3K, 4.5K, 7KHz.
- **Peak detectors:** Each filter output feeds a diode peak detector with ~50ms decay time. This gives a DC voltage proportional to energy in each band.
- **Comparator array:** Compares adjacent band energies and checks for suspicious gaps. Real voices: smooth roll-off, adjacent bands within 6dB. AI clones: sharp dips between bands, especially 2-4kHz.
- **Decision logic:** If more than 2 adjacent-band ratios exceed threshold, a simple logic gate (74HC08 AND) triggers the RED LED. Otherwise, GREEN stays lit.

Key Design Decisions

- **Why 8 bands, not 16?** More bands = more components but diminishing returns. 8 octave-spaced bands capture the spectral envelope accurately enough to detect synthesis artifacts while keeping the circuit on a single PCB.
- **Why Sallen-Key topology?** Unity-gain Sallen-Key filters use only 2 resistors and 2 caps per stage. Low component count, good stability, and the op-amp doesn't need to provide gain (reduces noise).
- **Why analog, not digital?** An MCU with FFT could do this, but the point is demonstrating that signal processing IS circuit design. In an interview, explaining why you chose each filter's Q factor and center frequency is far more impressive than 'I used an FFT library.'

Bill of Materials

Component	Part / Spec	Qty	~Cost
Electret microphone	CMA-4544PF-W	1	\$0.80
Quad op-amps	TL074 (4 channels each)	2	\$1.60
Resistors (1%)	Assorted values for filters	~40	\$2.00
Ceramic capacitors	Assorted values for filters	~20	\$1.50
Schottky diodes	BAT54 (for peak detectors)	8	\$1.00
Comparators	LM339 (quad comparator)	2	\$1.20
Logic gates	74HC08 (quad AND)	1	\$0.50
LEDs	Green + Red, 5mm	2	\$0.20

Protoboard / PCB	70x90mm	1	\$3.00
Power	9V battery + 7805 regulator	1	\$1.50

Build Steps

Mic + Pre-amp: Solder the electret mic with a 2.2K pull-up to VCC. Buffer through a JFET source follower. Feed into the first TL074 channel configured as a non-inverting amp with gain of 20 ($R_2/R_1 = 19$). Add a 10uF coupling cap on the output.

Filter Bank: Build 8 Sallen-Key bandpass filters. For each: calculate R and C values using $f_c = 1/(2\pi\sqrt{R_1 \cdot R_2 \cdot C_1 \cdot C_2})$ with $Q \approx 0.707$. Use 1% resistors. Wire each filter's input to the pre-amp output in parallel.

Peak Detectors: Each filter output → BAT54 Schottky diode → 10nF cap to ground with 100K discharge resistor. This gives ~50ms decay. The DC voltage across the cap represents the energy in that band.

Comparator Network: Use LM339 comparators. Compare adjacent band peak voltages through a resistive divider that sets the 6dB threshold. If band N is more than 6dB below band N-1 AND band N+1, that comparator output goes high.

Decision + Output: Feed comparator outputs into 74HC08 AND gates. If 2+ adjacent gaps detected simultaneously, the AND output goes high → RED LED. Otherwise, GREEN LED stays on through an inverter. Test with real speech vs. AI samples from ElevenLabs.

Interview Questions:

- Why did you choose 8 bands instead of using a digital FFT?
- How did you calculate the center frequencies and Q values for each filter?
- What determines the peak detector decay time, and how does it affect detection accuracy?
- What spectral characteristics distinguish AI-generated audio from real speech?
- How would you improve this design to reduce false positives?
- Could this circuit be defeated by a more advanced AI model? What would you change?

PROJECT 2

EMI Side-Channel Sniffer

Read electromagnetic leakage from a processor to determine what software it's running

Cost: ~\$30

Difficulty: Advanced

Build Time: 1-2 weekends

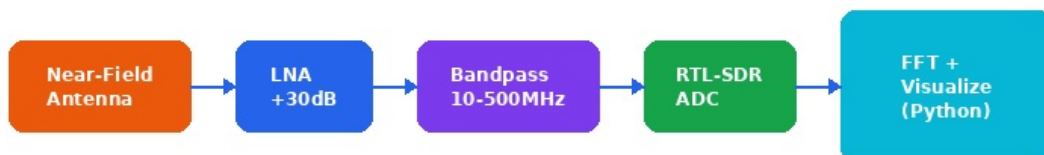
Why This Matters

Every processor emits electromagnetic radiation proportional to its switching activity. Different instructions draw different currents, creating unique EM signatures. Published research (Batina et al.) has demonstrated that you can extract neural network architectures from EM emissions alone. This project builds the capture tool: a near-field probe, low-noise amplifier, and SDR-based analysis pipeline.

What this proves to a recruiter: You understand RF signal chain design, low-noise analog, EM compatibility, and DSP. This is the intersection of analog, RF, and security — exactly where the highest-paying EE jobs live.

System Architecture

EMI Side-Channel Sniffer — Architecture



Antenna captures EM leakage → LNA amplifies → Filter isolates processor band
→ SDR samples IQ data → Python FFT reveals instruction-dependent spectral signatures

How It Works

- **Near-field H-probe:** A small loop antenna (10-20mm diameter) made from semi-rigid coax. Captures magnetic field emissions within a few cm of the target IC. The small aperture gives spatial selectivity — you can probe individual chips.
- **Low-noise amplifier (LNA):** A 30dB gain LNA (SPF5189Z or MAR-6+) with 50Ω input impedance. NF < 1dB. Powered via bias tee from the SDR or external 5V. Critical: proper input matching and bypass caps or you amplify noise, not signal.
- **Bandpass filter:** 10-500MHz range covers most digital switching activity. A simple LC filter or a commercial SAW filter removes out-of-band interference (WiFi, cellular, broadcast FM).
- **RTL-SDR:** The RTL2832U-based SDR dongle samples the filtered signal. 8-bit ADC, up to 2.4 MSPS bandwidth. Connect via USB to a laptop running Python.
- **Analysis (Python):** Use pyrtlsdr to capture IQ samples. Run spectrogram (short-time FFT) and correlate spectral patterns with known instruction sequences. Different AES rounds, different ML inference stages, idle vs. active — all have distinct signatures.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Semi-rigid coax	RG-402 or RG-405 (for probe)	30cm	\$3.00
SMA connectors	Edge-mount SMA female	3	\$2.00
LNA module	SPF5189Z (20MHz-6GHz, NF 0.6dB)	1	\$4.00
RTL-SDR dongle	RTL2832U + R820T2	1	\$12.00
Bypass capacitors	100nF + 10uF ceramic	6	\$0.50
SMA cables	Male-to-male, 30cm	2	\$3.00
Protoboard	For LNA mounting + filter	1	\$2.00
Power	USB 5V or battery pack	1	\$2.00

Build Steps

Build the H-probe: Strip 20mm of semi-rigid coax. Form the inner conductor into a loop (10-15mm diameter) and solder the tip back to the outer shield. This creates a shielded magnetic loop antenna. Terminate one end with SMA. The shield gap at the loop tip determines spatial resolution.

LNA stage: Mount SPF5189Z module on protoboard with clean ground plane. Add 100nF + 10uF bypass caps on VCC. Input from probe via SMA. Output to filter via SMA. Power from 3.3-5V through a ferrite bead to reduce power supply noise.

Bandpass filter: Build a 3rd-order LC bandpass (10-500MHz) using standard inductors and capacitors. Or use a commercial mini-circuits filter (SLP-550+). The goal is rejecting broadcast FM

(88-108MHz) and cellular bands while passing processor harmonics.

Connect to SDR: LNA output → filter → RTL-SDR input. Install pyrtlsdr in Python. Capture IQ data at 2.4 MSPS. Run waterfall spectrogram: you should see distinct spectral lines when you bring the probe near a running Raspberry Pi or Arduino.

Analyze: Write a Python script that captures 100ms windows and computes spectrograms. Run different code on the target (idle, AES encryption, neural net inference) and compare spectral signatures. Document the correlation between code execution and EM signature.

Interview Questions:

- Why a magnetic loop probe instead of an electric field probe?
- How does the LNA's noise figure affect your measurement sensitivity?
- What determines the spatial resolution of your near-field probe?
- How do you distinguish processor emissions from environmental RF interference?
- What countermeasures could a chip designer use against this type of attack?
- What sampling rate do you need to capture a 200MHz processor's emissions?

PROJECT 3

Hardware AI Kill Switch

Analog watchdog that physically disconnects AI hardware when power signature deviates

Cost: ~\$18

Difficulty: Intermediate

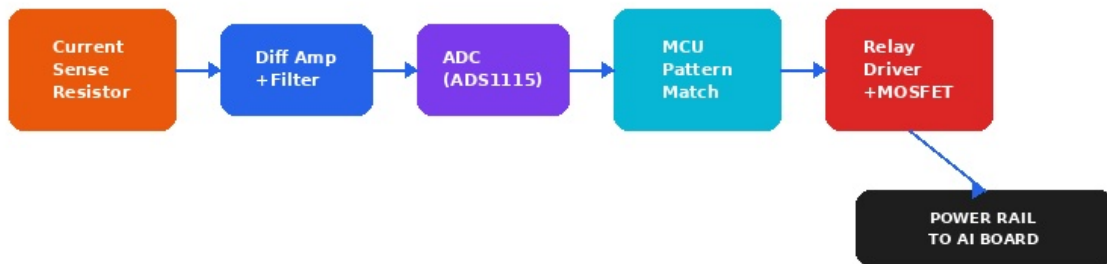
Build Time: 1-2 weekends

Why This Matters

Software safety mechanisms can be overridden by software. A hardware kill switch cannot. This project monitors the real-time power draw pattern of an AI compute board (Jetson Nano, Coral TPU, etc.) and compares it against expected inference signatures. If the power pattern deviates beyond a threshold — suggesting unexpected computation — a relay physically cuts the power rail. No firmware can override a relay that has already opened the circuit.

System Architecture

Hardware AI Kill Switch — Control Loop



Normal: relay stays closed, AI board runs | Anomaly: relay opens, hard disconnect
Software cannot override — power is physically cut before firmware can respond

How It Works

- **Current sense resistor:** A 10mΩ shunt resistor in series with the AI board's power rail. Produces a small voltage proportional to current draw. At 3A peak: 30mV full scale.
- **Differential amplifier + filter:** INA219 or discrete diff-amp (gain of 100) amplifies the shunt voltage. A low-pass filter at 100Hz smooths high-frequency switching noise while preserving the inference power envelope.

- **ADC (ADS1115):** 16-bit ADC samples the amplified signal at 860 SPS. Gives 1ms resolution of the power waveform — enough to capture inference cycles.
- **MCU pattern matching:** An ATmega328 or ESP32 compares the real-time power waveform against a stored 'expected' template using cross-correlation. If correlation drops below threshold for >500ms, it triggers the relay.
- **Relay driver:** A logic-level MOSFET (IRLZ44N) drives a 10A automotive relay. Normally closed — power flows to AI board. On trigger, MOSFET turns off, relay opens, power rail is physically cut. A manual reset button is required to reconnect.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Current sense resistor	10mΩ 1% (CSR series)	1	\$0.50
INA219	I2C current/power monitor	1	\$2.00
ADS1115	16-bit ADC module	1	\$3.00
ESP32 or ATmega328	MCU for pattern matching	1	\$3.00
IRLZ44N	Logic-level N-MOSFET	1	\$0.80
10A relay	Automotive relay, NO/NC	1	\$2.00
Flyback diode	1N4007 (across relay coil)	1	\$0.10
Push button	Momentary, for manual reset	1	\$0.30
Protoboard / PCB	70x90mm	1	\$3.00
Power	Barrel jack + input from supply	1	\$1.50

Build Steps

Power rail setup: Wire the 10mΩ shunt in series with the AI board's VCC line. Route both sides of the shunt to the INA219's IN+ and IN- pins. This is a Kelvin connection — run separate sense wires directly to the resistor pads to avoid measuring wire resistance.

Amplifier + ADC: Connect INA219 via I2C to the ESP32. Alternatively, build a discrete diff-amp with gain of 100 and feed into ADS1115. Set ADC to continuous mode at 860 SPS. Verify you see clean current waveforms when the AI board runs inference.

Template capture: Run the AI board with its normal inference workload. Record 10 seconds of current waveform at 860 SPS. Compute the average power envelope per inference cycle. Store this as the 'expected' template in ESP32 flash.

Correlation engine: Implement sliding-window cross-correlation in firmware. Compare each 500ms window of live data against the stored template. Output a correlation score (0-1). If score drops below 0.6 for more than 500ms continuously, trigger the relay.

Relay + reset: Wire ESP32 GPIO → IRLZ44N gate (with 10K pull-down). MOSFET drain → relay coil. 1N4007 flyback diode across relay coil. Relay contacts in series with AI board power. Normally energized = closed. Trigger = MOSFET off = relay opens = power cut. Add a manual push-button that resets the ESP32 to re-arm.

Interview Questions:

- Why use a physical relay instead of a solid-state switch?
- How does the Kelvin connection improve your current measurement accuracy?
- What determines your correlation threshold — how did you calibrate it?
- What's the maximum latency between anomaly detection and power cutoff?
- How would you prevent the AI board from detecting and gaming the power monitor?
- What failure modes exist in this design, and how would you mitigate them?

PROJECT 4

Deepfake-Proof Identity Badge

Wearable PCB with crypto element that signs live photo + timestamp + GPS

Cost: ~\$25

Difficulty: Advanced

Build Time: 1-2 weekends

Why This Matters

AI can now generate photorealistic images of anyone, anywhere. There is no software-only way to prove a photo is real. But hardware can. This badge uses a secure cryptographic element (ATECC608) to sign a hash of: live camera image + GPS coordinates + timestamp. The private key never leaves the chip — it physically cannot be extracted. The resulting QR code is cryptographic proof that a real person was physically at a specific location at a specific time.

System Architecture

Deepfake-Proof Identity Badge — Architecture



Camera captures live photo → Secure element signs (photo + timestamp + GPS hash)
→ E-paper shows QR code with cryptographic proof of physical presence

How It Works

- **Camera module:** OV2640 or ArduCAM Mini captures a 640x480 JPEG image. Connected to ESP32 via SPI.
- **GPS module:** NEO-6M provides latitude, longitude, and UTC timestamp. Connected via UART.
- **ATECC608B:** Microchip's secure element. Stores an ECC P-256 private key in tamper-resistant hardware. The key is generated on-chip and NEVER leaves. Signs a SHA-256 hash via I2C command. This is the same chip used in AWS IoT authentication.

- **ESP32:** Captures photo, reads GPS, computes SHA-256 hash of (image + lat/long + timestamp), sends hash to ATECC608 for signing, generates a QR code containing the signature + public key + metadata.
- **E-Paper display:** Shows the QR code. Anyone can scan it and verify the signature against the public key. If the image, location, or time has been tampered with, verification fails.

Bill of Materials

Component	Part / Spec	Qty	~Cost
ESP32-S3	DevKit or custom PCB	1	\$4.00
ATECC608B	Secure element (SOIC-8)	1	\$1.50
OV2640 camera	2MP, SPI interface	1	\$3.50
NEO-6M GPS	Module with antenna	1	\$5.00
E-Paper display	1.54" or 2.13" SPI	1	\$6.00
LiPo battery	3.7V 500mAh	1	\$3.00
Charge controller	TP4056 module	1	\$0.50
PCB	Custom 2-layer (JLCPCB)	5	\$2.00

Build Steps

Schematic + PCB: Design in KiCad. ESP32-S3 at center. ATECC608B on I2C bus (SDA/SCL with 4.7K pullups). OV2640 on SPI + XCLK/PCLK. NEO-6M on UART2. E-Paper on SPI2. TP4056 + LiPo for power. Add a tactile button for 'capture + sign' trigger.

ATECC608 provisioning: Use Microchip's CryptoAuthLib to generate a key pair on the secure element. The private key is generated inside the chip and cannot be read out. Export the public key and store it on a verification server or embed it in the QR code.

Capture pipeline: On button press: ESP32 captures JPEG from OV2640 → reads GPS fix from NEO-6M → concatenates (image_bytes + lat + lon + UTC_timestamp) → computes SHA-256 hash → sends hash to ATECC608 for ECC-P256 signature → receives 64-byte signature.

QR code generation: Encode (public_key + signature + lat + lon + timestamp + image_hash) into a QR code. Display on e-paper. The QR code is self-contained — anyone with the public key can verify the signature offline.

Verification: Write a simple Python or web script that scans the QR code, extracts the signature and metadata, and verifies the ECC signature against the public key. If any field was modified, verification fails. Demo: take a photo, show QR, verify on phone.

Interview Questions:

- Why ECC P-256 instead of RSA for the signature?
- What makes the ATECC608's key storage tamper-resistant at the hardware level?
- How do you handle GPS spoofing as an attack vector?
- What happens if the battery dies mid-signature? How do you handle partial state?
- How would you scale this to verify millions of badges — what's the PKI architecture?
- What's the difference between this and C2PA/Content Credentials?

PROJECT 5

Neuromorphic Sensor Fuser

Analog circuits that process sensor data the way neurons do — no clock, no ADC

Cost: ~\$12

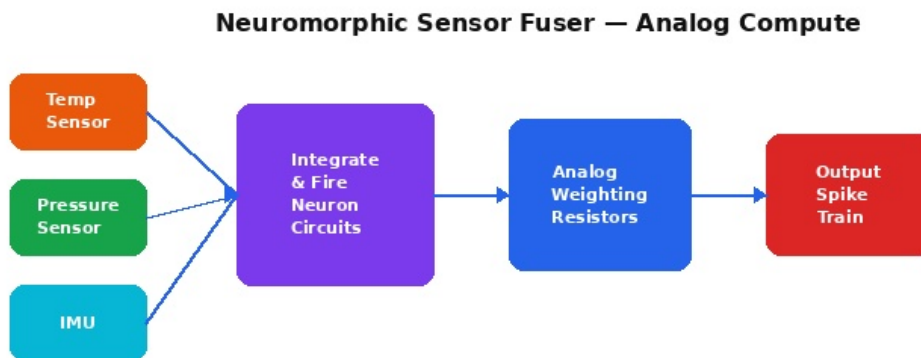
Difficulty: Advanced

Build Time: 1-2 weekends

Why This Matters

The future of AI hardware is moving away from digital clocked computation toward analog, event-driven processing. Neuromorphic computing mimics biological neurons: instead of sampling signals at a fixed clock rate, circuits fire 'spikes' only when input changes. This is massively more power-efficient and is the basis for chips like Intel's Loihi and IBM's TrueNorth. This project builds the fundamental building block: an analog integrate-and-fire neuron that fuses multiple sensor inputs.

System Architecture



No clock. No ADC. Continuous-time analog computation.

Sensors → RC integrate-and-fire circuits → weighted summation → spike output

How It Works

- **Sensor inputs:** Three analog sensors (temperature, pressure, IMU) produce continuous voltage signals. No ADC — the analog voltages feed directly into the neuron circuits.
- **Integrate-and-fire neuron:** An RC circuit integrates the input current. When the capacitor voltage reaches a threshold (set by a comparator), the circuit 'fires' — producing a short pulse — and the capacitor resets via a discharge MOSFET. Higher input = faster firing rate. This is exactly how biological neurons encode signal strength as spike frequency.

- **Synaptic weights:** Resistors between sensor outputs and neuron inputs act as analog weights. A 10K resistor = weak connection. A 1K resistor = strong connection. Changing resistor values changes how much each sensor influences the output — this is the analog equivalent of neural network weights.
- **Output layer:** A second integrate-and-fire neuron receives weighted spikes from the first layer. Its firing rate represents the fused decision. Connect to an LED or frequency counter to visualize the output spike train.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Temperature sensor	LM35 (analog output)	1	\$1.00
Pressure sensor	BMP180 analog out or MPX4115	1	\$2.50
IMU	ADXL335 (analog 3-axis)	1	\$3.00
Op-amps	LM324 (quad) for comparators	2	\$1.00
MOSFETs	2N7000 (for cap discharge)	4	\$0.60
Capacitors	1uF - 10uF (integration caps)	4	\$0.50
Resistors (1%)	Assorted for weights + thresholds	~30	\$1.50
LEDs + resistors	For spike visualization	4	\$0.40
Protoboard	70x90mm	1	\$2.00

Build Steps

Sensor front end: Wire LM35, BMP180 (or MPX4115 analog out), and ADXL335. Each produces 0-5V proportional to its measurement. Verify outputs with a multimeter before connecting to the neuron circuits.

Single neuron: Build one integrate-and-fire circuit: sensor output → weighting resistor → capacitor (integration). Comparator (LM324) monitors cap voltage against a threshold set by a voltage divider. When threshold is reached, comparator output goes high → triggers 2N7000 MOSFET → discharges cap → comparator resets. The output is a pulse train whose frequency encodes the input strength.

Add weights: Connect all three sensors to one neuron through different resistor values. Temperature: 4.7K (strong weight). Pressure: 10K (medium). IMU: 22K (weak). Now the neuron fires at a rate determined by the weighted sum of all three inputs — this IS sensor fusion.

Second layer: Build a second neuron. Feed the first neuron's spike output into it through another weighting resistor. Add a second first-layer neuron with different weights. The second layer neuron fires based on the combined pattern — a 2-layer analog neural network.

Visualize: Connect each neuron's output to an LED. You'll see them blinking at different rates as you change sensor inputs. Use an oscilloscope or logic analyzer to measure exact spike frequencies. Plot frequency vs. input to demonstrate the encoding relationship.

Interview Questions:

- How does the RC time constant determine the neuron's firing rate?
- How do the resistive weights map to neural network weights mathematically?
- What's the advantage of spike-rate coding over amplitude coding?
- How does power consumption compare to a digital MCU doing the same sensor fusion?
- What limits the number of layers you can cascade in analog?
- How does this relate to commercial neuromorphic chips like Intel Loihi?

PROJECT 6

RF Fingerprinting Authenticator

Identify wireless devices by their unique manufacturing imperfections

Cost: ~\$35

Difficulty: Advanced

Build Time: 1-2 weekends

Why This Matters

Every WiFi and Bluetooth chip has a unique RF signature caused by manufacturing variations — slightly different oscillator frequencies, amplifier gain curves, and phase noise profiles. MAC addresses can be spoofed in software. RF fingerprints cannot — they're determined by the physics of the silicon. This project builds an RF front end that captures these signatures and matches them against known devices. This is active research at DARPA and is used in military IFF (Identification Friend or Foe) systems.

System Architecture

RF Fingerprinting Authenticator — Signal Chain



Captures raw IQ from WiFi/BT preamble → correlates against known device signatures
Manufacturing imperfections create unique RF fingerprint — impossible to spoof

How It Works

- **2.4GHz antenna:** A PCB patch antenna or quarter-wave monopole tuned to 2.4GHz. Captures WiFi/BT preamble transmissions from nearby devices.
- **LNA + Mixer:** SPF5189Z amplifies the signal. A mixer (with local oscillator from Si5351 clock generator) downconverts to an intermediate frequency suitable for SDR sampling.
- **I/Q demodulator:** Extracts in-phase and quadrature components of the signal. This preserves both amplitude AND phase information — the phase is where manufacturing differences are most

visible.

- **Dual ADC:** Two channels sample I and Q simultaneously. ADS1115 for slow capture or RTL-SDR for high-speed. The key is capturing the first few microseconds of a transmission preamble, where the oscillator startup transient reveals the device's unique signature.
- **Correlator:** Python or FPGA-based correlation engine compares captured preamble IQ data against a database of known device signatures. Uses cross-correlation peak and phase offset as matching criteria.

Bill of Materials

Component	Part / Spec	Qty	~Cost
RTL-SDR dongle	RTL2832U + R820T2 (2.4GHz capable)	1	\$12.00
LNA	SPF5189Z module	1	\$4.00
2.4GHz antenna	PCB patch or SMA monopole	1	\$2.00
Si5351	Clock generator (for LO)	1	\$3.00
Mixer	ADE-1 or SA612 (if manual IF)	1	\$3.00
SMA cables + connectors	Assorted	4	\$4.00
Protoboard	For RF front end	1	\$3.00
Shielding	Copper tape for enclosure	1	\$2.00

Build Steps

Antenna: Build a quarter-wave monopole for 2.4GHz: 31mm wire soldered to SMA connector with a ground plane (copper-clad board). Or design a PCB patch antenna in KiCad — this alone is a portfolio-worthy skill.

RF front end: Connect antenna → LNA → bandpass filter (2.4-2.5GHz). If using RTL-SDR directly: some dongles can tune to 2.4GHz natively (R820T2). If not: use Si5351 as LO + mixer to downconvert to the SDR's range.

Capture preamble: Write Python script using pyrtlsdr to capture bursts at 2.4 MSPS. Trigger on signal power exceeding a threshold. Save the first 50 microseconds of each burst — this is the preamble where startup transients are strongest.

Build signature database: Capture 100 preambles from each known device (your phone, laptop, IoT devices). Average the IQ data per device to create a template. Store templates as numpy arrays.

Match + identify: For each new captured preamble, compute normalized cross-correlation against all templates. The template with the highest correlation score identifies the device. Plot ROC curves to characterize false accept/reject rates. A well-designed system achieves >95% accuracy across 10+ devices.

Interview Questions:

- What physical properties of a transmitter create its unique RF fingerprint?
- Why capture the preamble specifically instead of the data payload?
- How does I/Q demodulation preserve information that amplitude-only detection misses?
- What's your false acceptance rate, and how would you improve it?
- How does temperature affect RF fingerprints, and how do you compensate?
- What's the difference between this and MAC address authentication?

These aren't Arduino projects with a ChatGPT wrapper.

These are circuits that exist because AI created problems only hardware can solve. Each one involves real analog design, real RF, real signal processing, real cryptographic hardware. The kind of depth that makes an interviewer stop asking questions and start having a conversation.

Build even one of these projects. Document it with measurements and oscilloscope screenshots. Put it on your GitHub with the README template from my Portfolio Playbook. You'll have something on your resume that nobody else has.

Hardware is the last line of defense.

Follow @1nska on Instagram for build logs, new guides, and projects that make recruiters ask follow-up questions.

— Enes (@1nska) • Electrical Engineer